

Angular 19 Complete Tutorial

From Zero to Full-Stack Hero

Tailored for the WorkBridge Project
Angular 19 · Bootstrap 5 · JSON Server · TypeScript

CHAPTERS INSIDE:

Components · Modules · Services · Routing · Forms · HTTP · Guards · Pipes · Directives · RxJS

Complete with Code Examples, Tips, and Real Project Usage

TABLE OF CONTENTS

1. What is Angular?

- What is Angular
- Why Angular
- Angular vs Others
- Angular 19 New Features

2. Project Setup & Structure

- Installation
- CLI Commands
- Folder Structure
- angular.json
- app.config.ts

3. TypeScript Basics for Angular

- Types & Interfaces
- Classes
- Decorators
- Generics
- Optional Fields

4. Components

- What is a Component
- Creating Components
- Standalone Components
- Component Lifecycle
- Input & Output

5. Templates & Data Binding

- Interpolation
- Property Binding
- Event Binding
- Two-way Binding
- Template Variables

6. Directives

- *ngIf / @if
- *ngFor / @for
- *ngSwitch
- ngClass
- ngStyle
- Custom Directives

7. Services & Dependency Injection

- What is a Service
- Creating Services
- Injecting Services
- providedIn root

8. HTTP Client & JSON Server

- HttpClient Setup
- GET POST PUT DELETE
- Error Handling
- Observables

9. Routing & Navigation

- Router Setup
- Routes
- RouterLink
- Route Parameters
- Lazy Loading
- Guards

10. Reactive Forms

- FormGroup & FormControl
- Validators
- Form Submission
- Error Messages
- Dynamic Forms

11. RxJS & Observables

- Observable Basics
- Operators
- Subject & BehaviorSubject
- subscribe()

12. Pipes

- Built-in Pipes
- Custom Pipes
- Async Pipe

13. Angular + Bootstrap 5

- Setup
- Grid System
- Components
- Responsive Design

14. Guards & Auth

- AuthGuard
- AdminGuard
- CanActivate

- localStorage

15. WorkBridge Project Patterns

- Using Services in Pages
- Auth Flow
- Dashboard Pattern
- Complete Page Examples

CHAPTER 1

What is Angular?

Angular is a powerful frontend framework built by Google. It helps you build large, fast, and maintainable web applications. In this chapter you will understand what Angular is, why it is used, and how it compares to other frameworks.

1.1 Understanding Angular

Angular is a **framework** — not just a library. This means it gives you everything you need to build a complete web app: routing, forms, HTTP calls, state management, and more — all built in.

Think of Angular like a **fully equipped kitchen**. React gives you just a frying pan. Angular gives you the entire kitchen with all appliances already set up.

NOTE

Angular is written in TypeScript. This means you get type safety, better IDE support, and fewer bugs.

1.2 Why Use Angular?

Advantages

- Everything built-in (router, forms, HTTP)
- TypeScript by default
- Great for large applications
- Strong structure and conventions
- Excellent CLI tools
- Good for teams

Disadvantages

- Steep learning curve
- More boilerplate than React/Vue
- Larger bundle size initially
- Complex for very small projects

1.3 Angular vs Other Frameworks

Feature	Angular	React	Vue
Type	Full Framework	Library	Progressive Framework
Language	TypeScript	JavaScript/JSX	JavaScript
Learning Curve	Steep	Medium	Easy
Routing	Built-in	React Router (external)	Vue Router (external)

Forms	Built-in (Reactive)	Formik/React Hook Form	Vuelidate
HTTP	HttpClient (built-in)	Axios/Fetch (external)	Axios (external)
Best For	Large enterprise apps	Dynamic UIs	Beginner-friendly apps

1.4 Angular 19 New Features

Angular 19 introduces important changes you are already using in your WorkBridge project:

Feature	Old Way	New Way (Angular 17+)	Status in Your Project
Components	NgModule based	Standalone (no modules)	You are using standalone
Control Flow	*ngIf, *ngFor directives	@if, @for blocks	Both still work
App setup	AppModule	app.config.ts + app.routes.ts	You have both files
Signals	RxJS everywhere	Signals for simpler state	Optional, RxJS still works
Bootstrap	main.ts bootstraps module	bootstrapApplication()	Already set up for you

WorkBridge Project

Your WorkBridge project is Angular 19 with standalone components. Every component you generated has `standalone: true`. This is the modern way and you are already using it correctly.

CHAPTER 2

Project Setup & Structure

Learn how to set up an Angular project, understand every file and folder, and use the Angular CLI like a professional.

2.1 Installing Angular

Before creating a project you need Node.js and the Angular CLI:

Terminal Commands

```
# Step 1: Install Node.js from nodejs.org (LTS version)
```

```
# Step 2: Install Angular CLI globally
```

```
npm install -g @angular/cli
```

```
# Step 3: Verify installation
```

```
node -v # should show v18+ or v20+
```

```
npm -v # should show 9+
```

```
ng version # should show Angular CLI version
```

```
# Step 4: Create new project
```

```
ng new my-project --routing true --style css
```

```
# Step 5: Navigate into project
```

```
cd my-project
```

```
# Step 6: Install Bootstrap
```

```
npm install bootstrap bootstrap-icons
```

```
# Step 7: Run the app
```

```
ng serve
```

2.2 Angular CLI — Most Used Commands

Command	What it does	Example
ng serve	Starts dev server at localhost:4200	ng serve
ng serve -o	Starts dev server and opens browser	ng serve -o
ng build	Builds app for production	ng build
ng g c name	Generates a new component	ng g c pages/home
ng g s name	Generates a new service	ng g s services/auth
ng g i name	Generates a new interface/model	ng g i models/user
ng g guard name	Generates a route guard	ng g guard guards/auth
ng g pipe name	Generates a new pipe	ng g pipe pipes/currency
ng lint	Checks code for errors	ng lint
ng test	Runs unit tests	ng test

TIP

Always use ng g (generate) commands to create files. They set up the correct structure automatically. Never create Angular files manually.

2.3 Project Folder Structure — WorkBridge

Your WorkBridge project folder structure:

Folder/File	What it contains	Important?
src/app/	All your application code lives here	YES - main folder
src/app/app.ts	Root component — your app starts here	YES
src/app/app.html	Root template — contains router-outlet	YES
src/app/app.routes.ts	All 20 route definitions	YES
src/app/app.config.ts	App configuration (providers)	YES

src/app/component/	Public page components (home, jobs, gigs)	YES
src/app/pages/	Auth, dashboard, admin pages	YES
src/app/shared/	Reusable components (navbar, sidebar, footer)	YES
src/app/services/	All 45 services for data	YES
src/app/models/	All 44 TypeScript interfaces	YES
src/app/guards/	auth.guard and admin.guard	YES
src/index.html	Main HTML file — rarely edited	Rarely
src/styles.css	Global CSS styles	YES
angular.json	Angular configuration (bootstrap paths)	YES
package.json	Project dependencies list	Rarely
db.json	JSON Server mock database	YES

2.4 Key Configuration Files

angular.json — Where you added Bootstrap paths:

angular.json (styles section)

```
"styles": [

  "node_modules/bootstrap/dist/css/bootstrap.min.css",

  "node_modules/bootstrap-icons/font/bootstrap-icons.css",

  "src/styles.css"

]
```

app.config.ts — Application providers:

src/app/app.config.ts

```
import { ApplicationConfig } from "@angular/core";

import { provideRouter } from "@angular/router";

import { provideHttpClient } from "@angular/common/http";
```

```
import { routes } from "./app.routes";

export const appConfig: ApplicationConfig = {

  providers: [

    provideRouter(routes), // enables routing

    provideHttpClient(), // enables HTTP calls

  ]

};
```

CHAPTER 3

TypeScript Basics for Angular

Angular uses TypeScript. You do not need to master it fully, but understanding these key concepts will make Angular much easier.

3.1 Types & Interfaces

TypeScript adds **types** to JavaScript. This catches bugs before your code runs:

TypeScript Types

```
// JavaScript (no types - risky)

let name = "Ahmed";

name = 123; // No error! Bug later...


// TypeScript (with types - safe)

let name: string = "Ahmed";

name = 123; // ERROR! Cannot assign number to string


// Common types

let age: number = 25;

let isLoggedIn: boolean = true;

let skills: string[] = ["Angular", "TypeScript"];

let user: User | null = null; // User OR null
```

Interfaces — This is exactly what your 44 models are:

Interface Example (your user.ts)

```
// This is what your models/user.ts looks like

export interface User {

  id?: string; // ? means optional (not required)
```

```
name: string; // required

email: string; // required

role: "user" | "admin"; // only these two values allowed

isVerified: boolean; // true or false only

createdAt: string; // text

}

// Using the interface

let currentUser: User;

let allUsers: User[] = []; // array of users

let oneUser: User | null = null; // User or nothing
```

3.2 Classes in TypeScript

Class Example

```
// A class has properties and methods

export class UserService {

  private url = "http://localhost:3000/users"; // private = only inside this class

  public name = "UserService"; // public = accessible anywhere

  constructor(private http: HttpClient) {} // dependency injection

  // Method that returns an Observable of User array

  findAll(): Observable<User[]> {

    return this.http.get<User[]>(this.url);
```

```
}

}
```

3.3 Decorators — The @ Symbol

Decorators are special labels starting with @ that add metadata to classes:

Decorator	Where used	What it does
@Component()	Component class	Marks class as an Angular component
@Injectable()	Service class	Marks class as injectable (can be used in constructor)
@Input()	Component property	Allows parent to pass data into component
@Output()	Component property	Allows component to send events to parent
@Pipe()	Pipe class	Marks class as a pipe (data transformer)

Component Decorator

```
@Component({

  selector: "app-home", // HTML tag: <app-home>

  standalone: true, // no NgModule needed

  imports: [CommonModule], // what this component needs

  templateUrl: "./home.html",

  styleUrls: ["./home.css"]

})

export class Home {

  // component logic here

}
```

3.4 Optional Fields (?) — Used in All Your Models

Optional Fields

```
export interface Job {  
  
  id?: string; // ? = optional, can be undefined  
  
  title: string; // no ? = required, must have value  
  
}  
  
// When creating (no id needed – JSON Server generates it)  
  
const newJob: Job = {  
  
  title: "Angular Developer" // id is optional, so not required  
  
};  
  
// When reading (id will be there from server)  
  
const existingJob: Job = {  
  
  id: "abc-123",  
  
  title: "Angular Developer"  
  
};
```

TIP

All your 44 models use `id?: string` because JSON Server auto-generates the ID when you POST a new record. That is why `id` is optional.

CHAPTER 4

Components

Components are the building blocks of Angular. Every page, every card, every button you see is a component. Master components and you master Angular.

4.1 What is a Component?

A component is a **self-contained piece of UI** with its own HTML, CSS, and TypeScript logic. Think of it like a custom HTML tag that you define:

Part	File	Job
TypeScript (.ts)	home.ts	Logic: data, methods, variables
Template (.html)	home.html	View: what the user sees
Styles (.css)	home.css	Appearance: how it looks

Basic Component Structure

```
// home.ts – the TypeScript file

import { Component } from "@angular/core";

import { CommonModule } from "@angular/common";

@Component({
  selector: "app-home", // use as <app-home> in HTML

  standalone: true,

  imports: [CommonModule], // import modules needed

  templateUrl: "./home.html",

  styleUrls: ["./home.css"]

})

export class Home {

  title = "Welcome to WorkBridge"; // property
```

```
count = 0; // property

increment() { // method

  this.count++;

}

}
```

4.2 Component Lifecycle Hooks

Angular components have a lifecycle — they are created, updated, and destroyed. You can hook into these events:

Hook	When it runs	Most common use
<code>ngOnInit()</code>	Once after component is created	Fetch data from service — MOST USED
<code>ngOnDestroy()</code>	Just before component is destroyed	Clean up subscriptions
<code>ngOnChanges()</code>	When <code>@Input()</code> values change	React to parent data changes
<code>ngAfterViewInit()</code>	After view is fully rendered	Access DOM elements

`ngOnInit` — Most Important Lifecycle Hook

```
import { Component, OnInit, OnDestroy } from "@angular/core";

import { JobService } from "../../services/job";

import { Job } from "../../models/job";

@Component({

  selector: "app-jobs-list",

  standalone: true,

  imports: [CommonModule],
```



```
templateUrl: "./jobs-list.html"

}))

export class JobsList implements OnInit {

  jobs: Job[] = []; // empty array to start

  loading = true; // show spinner while loading

  constructor(private jobService: JobService) {}

  ngOnInit() {

    // This runs when component loads – perfect for fetching data

    this.jobService.findAll().subscribe({

      next: (data) => {

        this.jobs = data; // store fetched data

        this.loading = false; // hide spinner

      },

      error: (err) => {

        console.error(err);

        this.loading = false;

      }

    });

  }

}
```

TIP

Always fetch data inside `ngOnInit()`, not in the constructor. The constructor is for dependency injection only.

4.3 @Input() and @Output() — Component Communication

@Input() — Parent passes data INTO child component:

@Input() Example

```
// job-card.ts (CHILD component)

import { Component, Input } from "@angular/core";

import { Job } from "../../models/job";

@Component({
  selector: "app-job-card",
  standalone: true,
  template: `
    <div class="card">
      <h5>{{ job.title }}</h5>
      <p>{{ job.companyName }}</p>
    </div>
  `
})

export class JobCard {

  @Input() job!: Job; // receives job data from parent

}
```

```
// In PARENT component HTML

<app-job-card [job]="selectedJob"></app-job-card>

<app-job-card *ngFor="let j of jobs" [job]="j"></app-job-card>
```

@Output() — Child sends events UP to parent:

@Output() Example

```
// job-card.ts (CHILD component)

import { Component, Input, Output, EventEmitter } from "@angular/core";

@Component({...})

export class JobCard {

  @Input() job!: Job;

  @Output() applyClicked = new EventEmitter<Job>(); // event emitter

  onApply() {

    this.applyClicked.emit(this.job); // send job data to parent

  }

}

// In PARENT component HTML

<app-job-card [job]="j" (applyClicked)="handleApply($event)"></app-job-card>

// In PARENT component TS

handleApply(job: Job) {

  console.log("User wants to apply to:", job.title);
```

```
}
```

CHAPTER 5

Templates & Data Binding

Data binding connects your TypeScript data to your HTML template. It is how your app becomes dynamic and interactive.

5.1 The 4 Types of Data Binding

Type	Syntax	Direction	Example
Interpolation	{{ value }}	TS → HTML	{{ user.name }}
Property Binding	[property]="value"	TS → HTML	[src]="user.photo"
Event Binding	(event)="method()"	HTML → TS	(click)="login()"
Two-way Binding	[(ngModel)]="value"	Both ways	[(ngModel)]="email"

5.2 Interpolation — {{ }}

Displays TypeScript values in HTML:

Interpolation

```
// home.ts

export class Home {

  userName = "Ahmed Rahman";

  jobCount = 150;

  isLoggedIn = true;

  today = new Date();

}

// home.html

<h1>Welcome, {{ userName }}!</h1>

<p>We have {{ jobCount }} jobs available</p>
```

```
<p>Today is {{ today | date }}</p>
```

```
<p>2 + 2 = {{ 2 + 2 }}</p>
```

5.3 Property Binding — []

Dynamically sets HTML attributes and properties:

Property Binding

```
// home.ts

export class Home {

  imageUrl = "https://picsum.photos/300/200";

  isDisabled = false;

  buttonText = "Apply Now";

  cardClass = "card shadow-sm p-3";

}

// home.html

<img [src]="imageUrl" [alt]="buttonText">

<button [disabled]="isDisabled">{{ buttonText }}</button>

<div [class]="cardClass">Content</div>

<input [value]="userName">
```

5.4 Event Binding — ()

Listens to DOM events and calls TypeScript methods:

Event Binding

```
// login.ts
```

```
export class Login {

  onSubmit() { console.log("Form submitted"); }

  onKeyPress(event: KeyboardEvent) { console.log(event.key); }

  onHover(name: string) { console.log("Hovered:", name); }

}

// login.html

<button (click)="onSubmit()">Login</button>

<input (keyup)="onKeyPress($event)"> <!-- $event = the keyboard event -->

<div (mouseover)="onHover('card')">Hover me</div>

<form (ngSubmit)="onSubmit()">...</form>
```

5.5 Two-Way Binding — [()]

Used with forms. Updates both the variable AND the input simultaneously:

Two-Way Binding

```
// IMPORTANT: Must import FormsModule for ngModel

import { FormsModule } from "@angular/forms";

// register.ts

export class Register {

  email = "";

  password = "";

}
```

```
// register.html

<input [(ngModel)]="email" type="email" placeholder="Enter email">

<input [(ngModel)]="password" type="password">

<p>You typed: {{ email }}</p> <!-- updates live as you type -->
```

WARNING

For Reactive Forms (which your project uses), you do NOT use `[(ngModel)]`. Reactive Forms use `formControlName` instead. Two-way binding is for Template-Driven Forms only.

CHAPTER 6

Directives

Directives are instructions in the HTML that tell Angular to do something special. They are the most used feature in Angular templates.

6.1 *ngIf — Show/Hide Elements

Shows or hides HTML based on a condition:

*ngIf Directive

```
// home.ts

export class Home {

  isLoggedIn = false;

  loading = true;

  error = "";

}

// home.html

<!-- Show only if logged in -->

<div *ngIf="isLoggedIn">

  <p>Welcome back!</p>

</div>

<!-- Show spinner while loading -->

<div *ngIf="loading" class="text-center">

  <div class="spinner-border text-primary"></div>
```

```
</div>

<!-- Show error if exists -->

<div *ngIf="error" class="alert alert-danger">{{ error }}</div>

<!-- if/else -->

<div *ngIf="isLoggedIn; else notLoggedIn">Logged in content</div>

<ng-template #notLoggedIn>Please login first</ng-template>
```

TIP

Angular 17+ introduced `@if` as a modern replacement for `*ngIf`. Both work — your project uses `*ngIf` which is perfectly fine.

@if (New Angular 17+ Syntax)

```
<!-- Angular 17+ new @if syntax (both work) -->

@if (isLoggedIn) {

  <p>Welcome back!</p>

} @else {

  <p>Please login</p>

}
```

6.2 *ngFor — Loop Through Lists

Repeats HTML for every item in an array. Used on almost every page:

***ngFor Directive**

```
// jobs-list.ts

export class JobsList {
```

```
jobs: Job[] = [];

}

// jobs-list.html

<div class="row">

  <div class="col-md-4" *ngFor="let job of jobs">

    <div class="card">

      <h5>{{ job.title }}</h5>

      <p>{{ job.companyName }}</p>

      <p>{{ job.city }}</p>

    </div>

  </div>

</div>

<!-- With index -->

<li *ngFor="let job of jobs; let i = index">

  {{ i + 1 }}. {{ job.title }}

</li>

<!-- With trackBy for performance -->

<div *ngFor="let job of jobs; trackBy: trackByJob">

  {{ job.title }}

</div>
```

trackBy Function

```
// In component TS – trackBy improves performance

trackByJob(index: number, job: Job): string {

  return job.id!; // return unique identifier

}
```

6.3 ngClass — Dynamic CSS Classes

ngClass Directive

```
// status-badge.ts

export class StatusBadge {

  status = "active";

  isHighlighted = true;

}

// HTML – Single class toggle

<div [ngClass]="isHighlighted ? 'highlighted' : 'normal'">...</div>

// HTML – Multiple class conditions

<span [ngClass]="{

  'badge bg-success': status === 'active',

  'badge bg-danger': status === 'banned',

  'badge bg-warning': status === 'pending'

}">{{ status }}</span>
```

```
// HTML – Array of classes

<div [ngClass]="['card', 'shadow-sm', isActive ? 'active' : '']">
```

6.4 ngStyle — Dynamic Inline Styles

ngStyle Directive

```
// HTML

<div [ngStyle]="{

  'background-color': user.status === 'active' ? 'green' : 'red',

  'font-size': '14px',

  'padding': '10px'

}">

  User status: {{ user.status }}

</div>


// For rating bar width

<div class="progress-bar" [ngStyle]="{'width': rating * 20 + '%'}"></div>
```

WorkBridge Project

Your StatusBadge shared component uses [ngClass] to apply different Bootstrap badge colors based on the status value. This is the most practical use of ngClass in WorkBridge.

CHAPTER 7

Services & Dependency Injection

Services are where your business logic lives. They fetch data, manage state, and share information between components. Your 45 services are the backbone of WorkBridge.

7.1 What is a Service?

A service is a TypeScript class that does work for your components. Components should focus on the UI — services should focus on the logic and data:

Services SHOULD do

- Fetch data from API
- Store and manage data
- Share data between components
- Business logic (calculate, validate)
- Auth state management

Services SHOULD NOT do

- Handle HTML rendering
- Manage CSS styles
- Display data directly
- Control routing (use Router instead)
- Manage template variables

7.2 Creating and Using a Service

Service Structure

```
// job.service.ts – generated by: ng g s services/job

import { Injectable } from "@angular/core";

import { HttpClient } from "@angular/common/http";

import { Observable } from "rxjs";

import { Job } from "../models/job";

@Injectable({

  providedIn: "root" // available everywhere in the app

})

export class JobService {
```

```
private url = "http://localhost:3000/jobs";

constructor(private http: HttpClient) {}

findAll(): Observable<Job[]> {

  return this.http.get<Job[]>(this.url);

}

findById(id: string): Observable<Job> {

  return this.http.get<Job>(`${this.url}/${id}`);

}

save(job: Job): Observable<Job> {

  return this.http.post<Job>(this.url, job);

}

update(id: string, job: Job): Observable<Job> {

  return this.http.put<Job>(`${this.url}/${id}`, job);

}

delete(id: string): Observable<void> {

  return this.http.delete<void>(`${this.url}/${id}`);

}

}
```

7.3 Dependency Injection — How to Use Services

Dependency Injection (DI) is Angular's way of giving your component the service it needs. You declare what you need in the constructor:

Using a Service in a Component

```
// jobs-list.ts

import { Component, OnInit } from "@angular/core";

import { CommonModule } from "@angular/common";

import { JobService } from "../../services/job"; // import

import { Job } from "../../models/job";

@Component({

  selector: "app-jobs-list",

  standalone: true,

  imports: [CommonModule],

  templateUrl: "./jobs-list.html"

})

export class JobsList implements OnInit {

  jobs: Job[] = [];

  loading = true;

  // Angular injects JobService automatically here

  constructor(private jobService: JobService) {}

  ngOnInit() {
```



```
this.jobService.findAll().subscribe({

  next: (jobs) => {

    this.jobs = jobs;

    this.loading = false;

  },

  error: (err) => {

    console.error(err);

    this.loading = false;

  }

});

}
```

7.4 Injecting Multiple Services

Multiple Services

```
// dashboard-home.ts – uses multiple services

export class DashboardHome implements OnInit {

  constructor(

    private auth: AuthService,

    private jobService: JobService,

    private orderService: OrderService,

    private notificationService: NotificationService,
```

```
private walletService: WalletService,  
  
private router: Router // Angular Router is also a service  
  
) {}  
  
}
```

TIP

providedIn: "root" means Angular creates ONE instance of the service for the whole app. All components that inject JobService get the SAME instance. This is called a Singleton pattern.

CHAPTER 8

HTTP Client & JSON Server

HttpClient is how Angular talks to a backend. In your project it talks to JSON Server. Later when Spring Boot is ready, you just change the URL.

8.1 Setting Up HttpClient

Already done in your app.config.ts:

HttpClient Setup

```
// app.config.ts – already set up in your project

import { provideHttpClient } from "@angular/common/http";

export const appConfig: ApplicationConfig = {

  providers: [

    provideRouter(routes),

    provideHttpClient(), // this enables HTTP in whole app

  ]

};
```

8.2 The 5 HTTP Methods

Method	HTTP Verb	What it does	Your Service Method
findAll()	GET /jobs	Get all records	http.get(url)
getById()	GET /jobs/123	Get one record	http.get(url/id)
save()	POST /jobs	Create new record	http.post(url, data)
update()	PUT /jobs/123	Replace entire record	http.put(url/id, data)
delete()	DELETE /jobs/123	Remove record	http.delete(url/id)

8.3 Making HTTP Calls — Complete Example

Complete HTTP Examples

```
// GET – Fetch all jobs

this.jobService.findAll().subscribe({

  next: (jobs: Job[]) => {

    this.jobs = jobs;

    console.log("Got", jobs.length, "jobs");

  },

  error: (err) => console.error("Failed:", err)

});


// GET – Fetch one job by ID

this.jobService.getById("abc-123").subscribe({

  next: (job: Job) => this.selectedJob = job,

  error: (err) => console.error(err)

});


// POST – Create new job

const newJob: Job = {

  title: "Angular Developer",

  employerId: "user-1",

  companyName: "TechCorp",

  city: "Dhaka",

  area: "Banani",
```

```
    salaryMin: 30000,

    salaryMax: 60000,

    jobType: "full-time",

    experienceRequired: "2 years",

    deadline: "2025-12-31",

    status: "open",

    isDeleted: false,

    createdAt: new Date().toISOString(),

    description: "Build web apps",

    companyLogo: ""

  };

  this.jobService.save(newJob).subscribe({

    next: (saved) => console.log("Job created with id:", saved.id),

    error: (err) => console.error(err)

  });

  // PUT – Update a job

  this.jobService.update("abc-123", updatedJob).subscribe({

    next: () => console.log("Job updated!"),

    error: (err) => console.error(err)

  });
```

```
// DELETE – Remove a job

this.jobService.delete("abc-123").subscribe({

  next: () => console.log("Job deleted!"),

  error: (err) => console.error(err)

});
```

8.4 JSON Server Query Parameters

JSON Server supports filtering, sorting, and searching via URL parameters:

JSON Server Query Syntax

```
// Filter by field – used in your additional service methods

GET /jobs?status=open

GET /jobs?city=Dhaka&status=open

GET /users?role=admin


// Search (contains)

GET /jobs?title_like=developer


// Greater/Less than

GET /jobs?salaryMin_gte=30000

GET /jobs?price_lte=100


// Sort

GET /jobs?_sort=createdAt&_order=desc
```

```
// Limit results

GET /jobs?_limit=10


// Combined

GET /jobs?status=open&city=Dhaka&_sort=createdAt&_order=desc
```

WorkBridge Project

All your additional service methods (findByCity, findByStatus, searchByTitle etc.) use these JSON Server query parameters. When you switch to Spring Boot, these become @RequestParam in Java controllers.

CHAPTER 9

Routing & Navigation

Routing lets users navigate between pages without reloading the browser. Your app has 20 routes already set up.

9.1 How Routing Works

Angular intercepts browser URL changes and loads the correct component without a full page reload:

Route Definitions

```
// app.routes.ts – your route definitions

import { Routes } from "@angular/router";

import { authGuard } from "../guards/auth-guard";

export const routes: Routes = [

  // Public routes – anyone can access

  {

    path: "", // URL: localhost:4200/

    loadComponent: () => import("../component/home/home").then(m => m.Home)

  },

  {

    path: "jobs", // URL: localhost:4200/jobs

    loadComponent: () => import("../component/jobs-list/jobs-list").then(m => m.JobsList)

  },

  // Protected routes – only logged in users

  {
```



```
path: "dashboard",

canActivate: [authGuard], // guard applied here

loadComponent: () => import("../pages/dashboard/dashboard-home/dashboard-home")

.then(m => m.DashboardHome)

},

// Route with parameter

{ path: "details/:type/:id", loadComponent: () => import(...) }

// Fallback – if no route matches

{ path: "**", redirectTo: "" }

];
```

9.2 router-outlet — Where Pages Load

The router-outlet is a placeholder in your HTML where Angular loads the matched component:

```
router-outlet

<!-- app.html – your root template -->

<router-outlet></router-outlet>

<!-- When user visits /jobs, Angular replaces router-outlet with: -->

<app-jobs-list></app-jobs-list>

<!-- When user visits /dashboard, Angular replaces with: -->

<app-dashboard-home></app-dashboard-home>
```

9.3 Navigation in HTML — RouterLink

RouterLink in HTML

```
<!-- Must import RouterLink in component imports array -->

<!-- Basic link -->

<a routerLink="/jobs">Browse Jobs</a>

<!-- Active class – highlights current page link -->

<a routerLink="/jobs" routerLinkActive="active">Jobs</a>

<!-- Link with parameters -->

<a [routerLink]="['/details', 'job', job.id]">View Job</a>

<!-- Result: /details/job/abc-123 -->

<!-- Exact match for home -->

<a routerLink="/" routerLinkActive="active"

[routerLinkActiveOptions]="{exact: true}">Home</a>
```

9.4 Navigation in TypeScript — Router

Programmatic Navigation

```
// Import Router in your component

import { Router } from "@angular/router";

@Component({...})

export class Login {
```

```
constructor(private router: Router) {}

onLoginSuccess() {

  this.router.navigate(['/dashboard']); // go to dashboard

}

goToJobDetails(jobId: string) {

  this.router.navigate(['/details', 'job', jobId]); // go to details

}

goBack() {

  this.router.navigate(['../']); // go up one level

}

}
```

9.5 Reading Route Parameters

Reading Route Parameters

```
// Details page reads :type and :id from URL

// URL: /details/job/abc-123

import { Component, OnInit } from '@angular/core';

import { ActivatedRoute } from '@angular/router';

@Component({...})
```

```
export class Details implements OnInit {

  type = ""; // will be "job" or "gig"

  id = ""; // will be the actual ID


  constructor(private route: ActivatedRoute) {}


  ngOnInit() {

    this.route.params.subscribe(params => {

      this.type = params["type"]; // "job" or "gig"

      this.id = params["id"]; // "abc-123"


      // Now fetch the correct data

      if (this.type === "job") {

        this.jobService.getById(this.id).subscribe(...);

      } else {

        this.gigService.getById(this.id).subscribe(...);

      }

    });

  }

}
```

9.6 Lazy Loading — Already in Your Project

Lazy loading loads code only when needed — making your app faster on first load:

Lazy Loading

```
// EAGER loading (bad for performance – loads everything upfront)

{ path: "jobs", component: JobsList }

// LAZY loading (good – loads only when user visits /jobs)

{

  path: "jobs",

  loadComponent: () => import("../component/jobs-list/jobs-list")

    .then(m => m.JobsList) // only downloads when needed

}
```

TIP

All your 20 routes already use `loadComponent` (lazy loading). This is why your app starts fast — it only downloads the Home page code initially. Other pages are downloaded on demand.

CHAPTER 10

Reactive Forms

Reactive Forms are the Angular way to handle forms with validation. Your Register, Login, and other auth pages use Reactive Forms.

10.1 Reactive Forms vs Template-Driven Forms

Reactive Forms

- Defined in TypeScript
- Easier to validate
- Better for complex forms
- More control
- Used in YOUR project

Template-Driven Forms

- Defined in HTML template
- Uses ngModel
- Simpler for basic forms
- Less control
- Not used in your project

10.2 Setting Up a Form

Reactive Form Setup

```
// login.ts

import { ReactiveFormsModule, FormBuilder, FormGroup, Validators } from "@angular/forms";

@Component({

  standalone: true,

  imports: [CommonModule, ReactiveFormsModule], // must import

  ...

})

export class Login {

  form: FormGroup; // the form
```

```
constructor(private fb: FormBuilder) {

  // Define form structure and validation rules

  this.form = this.fb.group({

    email: ["", [Validators.required, Validators.email]],

    password: ["", [Validators.required, Validators.minLength(6)]],

  });

}

// Shortcut to access form controls

get f() { return this.form.controls; }

onSubmit() {

  if (this.form.invalid) return; // stop if form is invalid

  const email = this.f["email"].value;

  const password = this.f["password"].value;

  // Do login...

}

}
```

10.3 Form HTML Template

Form HTML

```
<!-- login.html -->

<form [formGroup]="form" (ngSubmit)="onSubmit()">
```

```
<!-- Email field -->

<div class="mb-3">

<label>Email</label>

<input formControlName="email" type="email" class="form-control">

<!-- Show error only when user has touched the field -->

<div *ngIf="f['email'].touched && f['email'].errors">

<small *ngIf="f['email'].errors?.['required']" class="text-danger">

Email is required

</small>

<small *ngIf="f['email'].errors?.['email']" class="text-danger">

Invalid email format

</small>

</div>

</div>

<!-- Password field -->

<div class="mb-3">

<label>Password</label>

<input formControlName="password" type="password" class="form-control">

<div *ngIf="f['password'].touched && f['password'].invalid"

class="text-danger small">
```



```

    Min 6 characters required

</div>

</div>

<!-- Submit button – disabled when form is invalid -->

<button type="submit" class="btn btn-primary w-100"

[disabled]="form.invalid">

Login

</button>

</form>

```

10.4 Built-in Validators

Validator	What it checks	Error key
Validators.required	Field cannot be empty	errors?.['required']
Validators.email	Must be valid email format	errors?.['email']
Validators.minLength(6)	Must be at least 6 chars	errors?.['minlength']
Validators.maxLength(20)	Must be max 20 chars	errors?.['maxlength']
Validators.min(0)	Number must be ≥ 0	errors?.['min']
Validators.max(100)	Number must be ≤ 100	errors?.['max']
Validators.pattern(regex)	Must match regex pattern	errors?.['pattern']

Custom Password Validation

```

// Custom validation – password match

passwordsMatch(): boolean {

return this.form.get("password")?.value ===

```

```
this.form.get("confirmPassword)?.value;

}

// In template

<div *ngIf="!passwordsMatch()" class="text-danger small">

  Passwords do not match

</div>

// Submit only when form is valid AND passwords match

onSubmit() {

  if (this.form.invalid || !this.passwordsMatch()) return;

  // proceed...

}
```

CHAPTER 11

RxJS & Observables

RxJS is the library Angular uses for async operations. HttpClient returns Observables. Understanding Observables is key to working with data in Angular.

11.1 What is an Observable?

An Observable is like a **stream of data over time**. When you call a service method, it returns an Observable. Nothing happens until you **subscribe** to it:

Observable and Subscribe

```
// The service returns an Observable

findAll(): Observable<Job[]> {

  return this.http.get<Job[]>(this.url); // nothing sent yet!

}

// In component – nothing happens until you subscribe

const jobs$ = this.jobService.findAll(); // still nothing!

// Now it actually fires the HTTP request

jobs$.subscribe({

  next: (jobs) => this.jobs = jobs, // runs when data arrives

  error: (err) => console.error(err), // runs if error

  complete: () => console.log("done") // runs when stream ends

});
```

TIP

Think of an Observable like ordering food. The Observable is the order. subscribe() is when you actually place the order. Without subscribe, nothing happens.

11.2 Common RxJS Operators

Operator	What it does	Use case
map()	Transform each value	Change data shape
filter()	Keep only matching values	Filter results
tap()	Side effect without changing data	Logging, store to localStorage
catchError()	Handle errors	Show error message
switchMap()	Cancel previous, start new	Search as you type
forkJoin()	Wait for multiple observables	Load multiple data at once
take(1)	Take only first value then complete	One-time HTTP calls

RxJS Operators Examples

```
// map – transform data

this.jobService.findAll().pipe(

  map(jobs => jobs.filter(j => !j.isDeleted))

).subscribe(jobs => this.jobs = jobs);


// tap – side effect (store user after login)

this.auth.login(email, password).pipe(

  tap(user => this.auth.storeUser(user!))

).subscribe(...);


// catchError – handle errors gracefully

this.jobService.findAll().pipe(

  catchError(err => {

    this.error = "Failed to load jobs";
```

```
return []; // return empty array on error

})

).subscribe(jobs => this.jobs = jobs);
```

11.3 BehaviorSubject — Sharing State

A BehaviorSubject is an Observable that holds a current value and emits it to new subscribers immediately. Used in your AuthService:

BehaviorSubject

```
// In auth.service.ts

import { BehaviorSubject } from "rxjs";

export class AuthService {

  // Holds current user – null if not logged in

  private currentUser$ = new BehaviorSubject<User | null>(null);

  // Observable that components can subscribe to

  user$ = this.currentUser$.asObservable();

  login(email: string, password: string) {

    // After login, update the subject

    this.currentUser$.next(loggedInUser);

  }

  logout() {

    this.currentUser$.next(null); // clear user
```

```
}

}

// In navbar.ts – subscribe to user changes

ngOnInit() {

  this.auth.user$.subscribe(user => {

    this.currentUser = user;

  });

}
```

CHAPTER 12

Pipes

Pipes transform data for display in templates. They do not change the actual data — only how it is displayed.

12.1 Built-in Pipes

Pipe	What it does	Example	Output
date	Formats a date	{{ today date:'dd/MM/yyyy' }}	10/05/2025
currency	Formats as currency	{{ price currency:'BDT' }}	BDT 1,500.00
uppercase	Makes text uppercase	{{ name uppercase }}	AHMED
lowercase	Makes text lowercase	{{ name lowercase }}	ahmed
titlecase	Title Case each word	{{ name titlecase }}	Ahmed Rahman
number	Formats a number	{{ count number:'1.0-0' }}	1,500
percent	Shows as percentage	{{ 0.85 percent }}	85%
json	Shows object as JSON	{{ user json }}	{name: Ahmed...}
async	Unwraps Observable	{{ jobs\$ async }}	Auto subscribes
slice	Slices array/string	{{ jobs slice:0:3 }}	First 3 jobs

Pipes in Templates

```
<!-- Pipe usage in templates -->

<p>{{ createdAt | date:"dd MMM yyyy" }}</p>

<p>{{ salary | currency:"BDT ":"symbol":"1.0-0" }}</p>

<p>{{ userName | titlecase }}</p>

<!-- Chaining pipes -->

<p>{{ name | uppercase | slice:0:10 }}</p>

<!-- Async pipe – subscribes automatically -->
```

```
<div *ngFor="let job of jobs$ | async">

  {{ job.title }}

</div>
```

12.2 Creating a Custom Pipe

Custom Pipe Example

```
// Generate: ng g pipe pipes/time-ago

import { Pipe, PipeTransform } from "@angular/core";

@Pipe({

  name: "timeAgo", // used as {{ date | timeAgo }}

  standalone: true,

})

export class TimeAgoPipe implements PipeTransform {

  transform(value: string): string {

    const date = new Date(value);

    const now = new Date();

    const diffMs = now.getTime() - date.getTime();

    const diffDays = Math.floor(diffMs / (1000 * 60 * 60 * 24));

    if (diffDays === 0) return "Today";

    if (diffDays === 1) return "Yesterday";

    if (diffDays < 7) return `${diffDays} days ago`;
```



```
return date.toLocaleDateString();

}

}

// Use in template (after importing TimeAgoPipe)

<p>Posted {{ job.createdAt | timeAgo }}</p>

<!-- Output: "3 days ago" -->
```

WorkBridge Project

Useful pipes for WorkBridge: timeAgo (show "3 days ago" on job cards), currency with BDT symbol on wallet/orders, titlecase on user names, date formatting on deadlines and timestamps.

CHAPTER 13

Angular + Bootstrap 5

Bootstrap 5 gives you ready-made UI components. Combined with Angular, you can build professional responsive interfaces quickly.

13.1 Bootstrap Grid System

Bootstrap uses a 12-column grid. Divide the row into columns using col-X classes:

Bootstrap Grid

```
<!-- Full width -->

<div class="col-12">Full width</div>

<!-- Two equal columns -->

<div class="row">

  <div class="col-6">Left</div>

  <div class="col-6">Right</div>

</div>

<!-- Responsive: full on mobile, 4 cols on desktop -->

<div class="row">

  <div class="col-12 col-md-4" *ngFor="let job of jobs">

    {{ job.title }}

  </div>

</div>

<!-- Auto columns -->
```

```
<div class="row">

<div class="col">Auto</div>

<div class="col">Auto</div>

<div class="col">Auto</div>

</div>
```

13.2 Most Used Bootstrap Classes in WorkBridge

Category	Classes	Use case
Cards	card, card-body, card-title, shadow-sm	Job cards, gig cards, dashboard cards
Buttons	btn btn-primary, btn-outline-danger, btn-sm	Apply, Order, Delete buttons
Badges	badge bg-success, bg-warning, bg-danger	Status badges on orders/jobs
Alerts	alert alert-danger, alert-success	Error and success messages
Forms	form-control, form-label, form-select	All form inputs
Tables	table table-hover table-striped	Orders table, users table
Nav	nav nav-tabs, nav-link active	Dashboard tabs
Spinner	spinner-border text-primary	Loading indicator
Spacing	p-3 m-2 mb-4 px-3 py-2 gap-2	Padding and margin
Flexbox	d-flex justify-content-between align-items-center	Layout alignment

Bootstrap Job Card

```
<!-- Job Card using Bootstrap -->

<div class="card shadow-sm border-0 h-100" style="border-radius: 12px;">

<div class="card-body p-3">

<div class="d-flex align-items-center mb-3">

<img [src]="job.companyLogo" class="rounded-circle me-2" width="40">
```

```
<div>

<h6 class="mb-0 fw-semibold">{{ job.title }}</h6>

<small class="text-muted">{{ job.companyName }}</small>

</div>

</div>

<div class="d-flex gap-2 flex-wrap mb-3">

<span class="badge bg-light text-dark">

<i class="bi bi-geo-alt me-1"></i>{{ job.city }}

</span>

<span class="badge bg-primary">{{ job.jobType }}</span>

</div>

<p class="text-success fw-semibold mb-3">

&#x9F3;{{ job.salaryMin | number }} - &#x9F3;{{ job.salaryMax | number }}

</p>

<a [routerLink]="['/details', 'job', job.id]"

class="btn btn-outline-primary btn-sm w-100">

View Details

</a>

</div>

</div>
```

CHAPTER 14

Guards & Authentication

Guards protect routes. They check if a user should be allowed to access a page before Angular loads it.

14.1 How Guards Work

A guard is a function that returns true (allow) or false (block + redirect). Angular runs it before loading the page:

AuthGuard

```
// auth-guard.ts – your existing guard

import { CanActivateFn, Router } from "@angular/router";

import { inject } from "@angular/core";

import { AuthService } from "../services/auth";

export const authGuard: CanActivateFn = () => {

  const auth = inject(AuthService);

  const router = inject(Router);

  if (auth.isLoggedIn()) {

    return true; // allow access

  }

  router.navigate(["/login"]); // redirect to login

  return false; // block access

};
```

AdminGuard

```
// admin-guard.ts – your existing guard

export const adminGuard: CanActivateFn = () => {

  const auth = inject(AuthService);

  const router = inject(Router);

  if (auth.isAdmin()) {

    return true; // user is admin – allow

  }

  router.navigate(["/dashboard"]); // not admin – go to dashboard

  return false;

};
```

Applying Guards to Routes

```
// Applying guards in routes

// Dashboard – only logged in users

{

  path: "dashboard",

  canActivate: [authGuard],

  loadComponent: () => import(...)

},

// Admin – must be logged in AND must be admin

{
```

```
path: "admin",

canActivate: [authGuard, adminGuard],

loadComponent: () => import(...)

},
```

14.2 LocalStorage — Storing Auth State

LocalStorage Usage

```
// Store user after login

localStorage.setItem("workbridge_user", JSON.stringify(user));

// Read user

const data = localStorage.getItem("workbridge_user");

const user = data ? JSON.parse(data) : null;

// Remove user on logout

localStorage.removeItem("workbridge_user");

// Check if logged in

const isLoggedIn = localStorage.getItem("workbridge_user") !== null;
```

WARNING

LocalStorage is for frontend-only projects like yours (JSON Server). In production with Spring Boot, you will use HttpOnly cookies or session tokens for better security. LocalStorage is fine for learning.

CHAPTER 15

WorkBridge Project Patterns

This chapter shows real code patterns used in your WorkBridge project. These are the exact patterns you will use on every page.

15.1 The Standard Page Pattern

Every dashboard page follows this exact pattern:

Standard Page Pattern

```
// Standard page pattern – used on EVERY page

import { Component, OnInit } from "@angular/core";

import { CommonModule } from "@angular/common";

import { RouterLink } from "@angular/router";

import { Sidebar } from "../../../shared/sidebar/sidebar";

import { JobService } from "../../../services/job";

import { AuthService } from "../../../services/auth";

import { Job } from "../../../models/job";

@Component({

  selector: "app-jobs-management",

  standalone: true,

  imports: [

    CommonModule, // *ngIf *ngFor

    RouterLink, // routerLink

    Sidebar, // <app-sidebar>

  ],
```



```
templateUrl: "../jobs-management.html"

}))

export class JobsManagement implements OnInit {

  // 1. Data variables

  jobs: Job[] = [];

  loading = true;

  error = "";

  activeTab = "applied"; // for tab switching


  // 2. Inject services

  constructor(

    private jobService: JobService,

    private auth: AuthService

  ) {}


  // 3. Fetch data on init

  ngOnInit() {

    this.loadData();

  }


  loadData() {

    const userId = this.auth.getCurrentUserId(!);

    this.jobService.findByApplicantId(userId).subscribe({
```

```
next: (jobs) => {

  this.jobs = jobs;

  this.loading = false;

},

error: () => {

  this.error = "Failed to load jobs";

  this.loading = false;

}

});

}

// 4. Methods for user actions

deleteJob(id: string) {

  if (!confirm("Are you sure?")) return;

  this.jobService.delete(id).subscribe({

    next: () => this.jobs = this.jobs.filter(j => j.id !== id)

  });

}

}
```

15.2 Standard Dashboard HTML Layout

Dashboard HTML Layout

```
<!-- Standard dashboard page layout -->
```

```
<div class="d-flex">

  <!-- Sidebar -->

  <app-sidebar></app-sidebar>

  <!-- Main content -->

  <div class="flex-grow-1 p-4" style="background: #f8f9fa; min-height: 100vh;">

    <!-- Page header -->

    <div class="d-flex justify-content-between align-items-center mb-4">

      <h4 class="fw-bold mb-0">Jobs Management</h4>

      <button class="btn btn-primary btn-sm">

        <i class="bi bi-plus me-1"></i>Post a Job

      </button>

    </div>

    <!-- Loading spinner -->

    <div *ngIf="loading" class="text-center py-5">

      <div class="spinner-border text-primary"></div>

    </div>

    <!-- Error -->

    <div *ngIf="error" class="alert alert-danger">{{ error }}</div>
```

```
<!-- Content (only show when not loading) -->

<div *ngIf="!loading && !error">

<!-- Tabs -->

<ul class="nav nav-tabs mb-4">

<li class="nav-item">

<a class="nav-link" [class.active]="activeTab==='applied'"

(click)="activeTab='applied'">Applied Jobs</a>

</li>

<li class="nav-item">

<a class="nav-link" [class.active]="activeTab==='posted'"

(click)="activeTab='posted'">Posted Jobs</a>

</li>

</ul>

<!-- Tab content -->

<div *ngIf="activeTab==='applied'">

<!-- Applied jobs content -->

<div class="card shadow-sm border-0 p-3" *ngFor="let job of jobs">

<h6>{{ job.title }}</h6>

</div>

</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

15.3 Loading State Pattern

Loading State Pattern

```
// Always use 3 state variables

loading = true; // show spinner

error = ""; // show error message

data: Job[] = []; // the actual data

// In ngOnInit

this.service.findAll().subscribe({

  next: (result) => {

    this.data = result;

    this.loading = false;

  },

  error: (err) => {

    this.error = "Failed to load. Check JSON Server is running.";

    this.loading = false;

  }

});
```

```
<!-- In HTML -->

<div *ngIf="loading">Loading...</div>

<div *ngIf="error" class="alert alert-danger">{{ error }}</div>

<div *ngIf="!loading && !error">

<!-- show data -->

</div>
```

15.4 Complete Working Page — Home Page Skeleton

Home Page Component

```
// home.ts – complete working example

import { Component, OnInit } from "@angular/core";

import { CommonModule } from "@angular/common";

import { RouterLink } from "@angular/router";

import { Navbar } from "../../shared/navbar/navbar";

import { Footer } from "../../shared/footer/footer";

import { JobService } from "../../services/job";

import { GigService } from "../../services/gig";

import { Job } from "../../models/job";

import { Gig } from "../../models/gig";

@Component({

  selector: "app-home",

  standalone: true,
```

```
imports: [CommonModule, RouterLink, Navbar, Footer],

templateUrl: "./home.html"

})

export class Home implements OnInit {

  jobs: Job[] = [];

  gigs: Gig[] = [];

  loading = true;

  searchTerm = "";

  searchType = "jobs";

  constructor(

    private jobService: JobService,

    private gigService: GigService

  ) {}

  ngOnInit() {

    // Load featured jobs

    this.jobService.findOpenJobs().subscribe({

      next: (jobs) => {

        this.jobs = jobs.slice(0, 6); // show only 6

        this.loading = false;

      }

    })

  }

}
```

```

});

// Load featured gigs

this.gigService.findActiveGigs().subscribe({

  next: (gigs) => this.gigs = gigs.slice(0, 6)

});

}

}

```

15.5 Quick Reference — Imports Cheat Sheet

When building any page, these are the imports you will need:

What you need	Import from	Add to imports array
*ngIf, *ngFor, async pipe	@angular/common	CommonModule
routerLink, routerLinkActive	@angular/router	RouterLink, RouterLinkActive
Reactive Forms	@angular/forms	ReactiveFormsModule
Template Forms (ngModel)	@angular/forms	FormsModule
Navigate programmatically	@angular/router	Router (inject, not import)
Read route params	@angular/router	ActivatedRoute (inject)
Navbar component	../shared/navbar/navbar	Navbar
Sidebar component	../shared/sidebar/sidebar	Sidebar
Footer component	../shared/footer/footer	Footer
Status Badge	../shared/status-badge/status-badge	StatusBadge
Star Rating	../shared/star-rating/star-rating	StarRating

15.6 Common Mistakes & Fixes

Error / Problem	Likely Cause	Fix
-----------------	--------------	-----

Can't use *ngIf/*ngFor	CommonModule not imported	Add CommonModule to imports array
routerLink not working	RouterLink not imported	Add RouterLink to imports array
HTTP calls not working	provideHttpClient missing	Add provideHttpClient() to app.config.ts
Form not working	ReactiveFormsModule missing	Add ReactiveFormsModule to imports
Service not found	Wrong import path	Check relative path to services/
Page redirects to login	authGuard blocking — not logged in	Login first, or check localStorage
JSON Server error	JSON Server not running	Run: json-server --watch db.json --port 3000
CORS error	Port mismatch	Make sure Angular on 4200 and JSON Server on 3000
Cannot read property of null	Data not loaded yet	Use *ngIf or optional chaining user?.name
Model interface error	Missing required field	Check interface — are you providing all required fields?

15.7 Two Terminal Rule

You always need two terminals running simultaneously:

Two Terminals Required

```
# Terminal 1 – Angular dev server
```

```
cd C:/path/to/freelancing_cum_jobportal
```

```
ng serve
```

```
# App runs at: http://localhost:4200
```

```
# Terminal 2 – JSON Server (mock backend)
```

```
cd C:/path/to/freelancing_cum_jobportal
```

```
json-server --watch db.json --port 3000
```

```
# API runs at: http://localhost:3000

# Test your API endpoints in browser:

http://localhost:3000/users

http://localhost:3000/jobs

http://localhost:3000/gigs
```

TIP

Bookmark <http://localhost:3000> in your browser. You can view, add, edit, and see your mock data there. It is your database UI while developing.

15.8 Moving from JSON Server to Spring Boot Later

When your Spring Boot backend is ready, you only need to change the URL in each service. Nothing else changes:

Switching from JSON Server to Spring Boot

```
// NOW (JSON Server)

private url = "http://localhost:3000/jobs";

// LATER (Spring Boot)

private url = "http://localhost:8080/api/jobs";

// Everything else stays EXACTLY the same:

findAll(): Observable<Job[]> {

  return this.http.get<Job[]>(this.url); // same code!

}

// TIP: Use an environment file for easy switching
```

```
// src/environments/environment.ts

export const environment = {

  production: false,

  apiUrl: "http://localhost:3000" // change this one place

};

// Then in service:

private url = environment.apiUrl + "/jobs";
```

WorkBridge Project

Your WorkBridge project is perfectly structured for future Spring Boot integration. The 45 services with their 5 methods map directly to Spring Boot REST endpoints. When ready, just change the base URL in one place.

ANGULAR CHEAT SHEET — Quick Reference

Feature	Syntax	Purpose
Interpolation	{{ value }}	Display variable in HTML
Property Binding	[src]="imgUrl"	Set HTML attribute dynamically
Event Binding	(click)="method()"	Handle DOM events
Two-way	[(ngModel)]="val"	Sync input with variable
Conditional	*ngIf="condition"	Show/hide element
Loop	*ngFor="let x of list"	Repeat element for each item
Dynamic class	[ngClass]="{cls: bool}"	Apply class conditionally
Route link	routerLink="/path"	Navigate to route
Active link	routerLinkActive="active"	Add class when route matches
Route params	route.params.subscribe()	Read URL parameters
Navigate	router.navigate(['/path'])	Go to route from TypeScript
HTTP GET	http.get(url)	Fetch data from API
HTTP POST	http.post(url, data)	Create new record
HTTP PUT	http.put(url+/id, data)	Update record
HTTP DELETE	http.delete(url+/id)	Delete record
Subscribe	.subscribe({next, error})	Execute Observable
Lifecycle	ngOnInit()	Runs on component create
Input	@Input() name!: Type	Receive data from parent
Output	@Output() ev = new EventEmitter()	Send event to parent
Form group	fb.group({field: ['', rules]})	Create reactive form
Validator	Validators.required	Make field required
Form submit	(ngSubmit)="onSubmit()"	Handle form submission
Guard	canActivate: [authGuard]	Protect route
Pipe	{{ value date }}	Transform display value

You are building WorkBridge — a real production-level platform. Master these patterns and you can build anything.